

Appendix A

Computational Benchmark — Construct I

Full Protocol Details, Extended Results, and Source Code

This appendix provides technical details supplementing Section 5 of the main text. Readers familiar with Section 5 may proceed directly to A.1 (extended robustness results) or A.2 (source code). The benchmark reported here tests Construct I exclusively. Validation of Constructs II–V is designated as future work.

A.1 Extended Robustness Results

Section 5.4 of the main text reports primary results at $\tau = 10$, $\sigma = 0.10$, seed = 42. Tables A1–A3 below report the full multi-seed and multi-noise-level results.

Table A1. Classification accuracy across all seeds ($\tau = 10$, $\sigma = 0.10$).

System	Seed 42	Seed 123	Seed 456	Seed 789	Seed 2026	Positive fraction
Markov	–	–	–	–	–	0%
Hidden State	+	+	+	+	+	100%
Delay Line	+	+	+	+	+	100%
Recurrent	+	+	+	+	+	100%

Table A2. Mean relative improvement across seeds ($\tau = 10$, $\sigma = 0.10$).

System	Mean rel. improvement	SD
Markov	–0.033%	0.028%
Hidden State	+9.32%	0.259%
Delay Line	+32.87%	0.633%
Recurrent	+6.28%	0.367%

Table A3. Classification robustness across noise levels ($\sigma \in \{0.05, 0.10, 0.20, 0.30\}$, $\tau = 10$, seed 42).

System	$\sigma = 0.05$	$\sigma = 0.10$	$\sigma = 0.20$	$\sigma = 0.30$
Markov	–	–	–	–
Hidden State	+	+	+	+
Delay Line	+	+	+	+
Recurrent	+	+	+	+

No false positive was observed for Markov at any noise level or seed. The Markov confidence interval remained roughly three orders of magnitude narrower than the non-Markov systems across all conditions, consistent with a true null effect.

A.2 Capacity Control: Full Results

The placebo history condition maintains identical feature dimensionality but destroys temporal ordering via row-permutation. The gap between real and placebo improvement confirms that gains reflect temporal information content, not model capacity.

Table A4. Capacity control: real vs. placebo history improvement ($\tau = 10$, $\sigma = 0.10$, seed 42).

System	Real improvement	Placebo improvement	Difference
Markov	-0.012%	-0.017%	+0.005%
Hidden State	+9.601%	+0.029%	+9.572%
Delay Line	+31.937%	-0.043%	+31.980%
Recurrent	+6.746%	-0.060%	+6.806%

A.3 State Reconstruction Diagnostic: Full Results

The PCA-compressed history model uses $n = 3$ principal components fitted on training data only. The “residual advantage” column shows the percentage improvement of full history M_1 over the PCA-compressed model, isolating temporal structure not captured by low-dimensional state reconstruction under the specified model class.

Table A5. State reconstruction diagnostic (PCA, $n = 3$ components, $\tau = 10$, $\sigma = 0.10$, seed 42).

System	MSE (full history M_1)	MSE (PCA-compressed)	Residual advantage
Markov	0.010019	0.010019	0.000%
Hidden State	0.024292	0.026229	+7.38%
Delay Line	0.010109	0.011336	+10.8%
Recurrent	0.021251	0.022716	+6.45%

PCA compression to three components failed to recover the full predictive advantage for Hidden State, Delay Line, and Recurrent. The residual advantage indicates temporal structure not captured by low-dimensional state reconstruction. This does not establish a specific mechanism; it rules out one class of explanations (dimensionality-reducible state reconstruction) under the pre-specified comparison.

A.4 History Dependence Profile

The history dependence profile $\Delta L(\tau)$ across horizon lengths $\tau \in \{1, 2, 3, 5, 10\}$ reveals at which temporal scales historical information is most predictive. This profile is informative in its own right and should be reported in empirical applications alongside the primary classification.

Table A6. Relative improvement $\Delta L(\tau)$ by history horizon ($\sigma = 0.10$, seed 42).

System	$\tau = 1$	$\tau = 2$	$\tau = 3$	$\tau = 5$	$\tau = 10$
Markov	-0.003%	-0.005%	-0.008%	-0.010%	-0.012%
Hidden State	+3.21%	+6.47%	+8.02%	+8.94%	+9.60%
Delay Line	+0.08%	+0.11%	+28.4%	+31.1%	+31.9%
Recurrent	+2.18%	+4.63%	+5.89%	+6.52%	+6.75%

The Delay Line profile shows a sharp transition at $\tau = 3$, consistent with the ground-truth lag-3 dependence. Hidden State and Recurrent show gradual monotonic increases consistent with exponentially decaying latent state influence. The profile structure is itself a diagnostic signal and should be reported alongside primary classifications in empirical applications.

A.5 Source Code

The complete source code for the Construct I benchmark is available at the author's Zenodo repository (released upon publication). The implementation includes:

- System generators for all four ground-truth architectures (Markov, Hidden State, Delay Line, Recurrent) with configurable noise levels and random seeds.
- Supervised dataset construction with configurable burn-in (100 steps) and trajectory length (500 steps).
- Trajectory-blocked 5-fold cross-validation for Ridge regression regularization parameter selection on training data only.
- Paired permutation test (5000 permutations) and paired bootstrap (2000 replications) with trajectory as the resampling unit.
- Placebo capacity control via row-permutation of historical feature blocks.
- PCA state reconstruction diagnostic with components fitted on training data only.
- History dependence profile computation across $\tau \in \{1, 2, 3, 5, 10\}$.
- Structured results output with classification labels, confidence intervals, p-values, and effect sizes.

All pre-registered parameters (trajectory counts, history windows, noise levels, seeds, classification criteria) were fixed before data generation. No parameters were modified after observing results.

Code:

```
# =====
# HDD BENCHMARK v1.0
# History-Dependent Dynamics
#
# CONSTRUCT I
# History-Dependent Predictive Structure
#
# Goal:
# Test whether history  $H_t$  improves prediction of  $X_{t+1}$ 
# beyond the observed state  $X_t$  and the current input  $P_t$ .
#
# IMPORTANT:
# This benchmark does NOT test Constructs II-V.
# It is a computational validation of Construct I.
#
# Benchmark author: reproducible implementation
# Target platform: Google Colab
```

```

# Python >= 3.9
# =====

# =====
# 0. IMPORTS
# =====

import os
import sys
import json
import math
import time
import platform
import warnings
from dataclasses import dataclass, asdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA

warnings.filterwarnings("ignore")

print("=" * 70)
print("HDD BENCHMARK v1.0")
print("Construct I \u2014 History-Dependent Predictive Structure")
print("=" * 70)

# =====
# 1. PRE-REGISTERED CONFIGURATION
# =====

@dataclass
class Config:

    # -----
    # Data
    # -----
    n_train_trajectories: int = 120
    n_test_trajectories: int = 60

    trajectory_length: int = 500
    burn_in: int = 100

    # Maximum history tested
    max_history: int = 10

    # -----
    # Noise
    # -----
    noise_std: float = 0.10

    # -----
    # Modeling
    # -----
    alpha_grid: tuple = (
        1e-4,
        3e-4,
        1e-3,
        3e-3,
        1e-2,
        3e-2,
        1e-1,
    )

```

```

        3e-1,
        1.0,
        3.0,
        10.0,
        30.0,
        100.0
    )

    # number of inner folds
    inner_folds: int = 5

    # -----
    # Inference
    # -----
    bootstrap_reps: int = 2000
    permutation_reps: int = 5000

    alpha_stat: float = 0.05

    # minimum relevant effect
    # 1% relative improvement
    practical_relative_threshold: float = 0.01

    # -----
    # Robustness
    # -----
    random_seed: int = 42

    robustness_seeds: tuple = (
        42,
        123,
        456,
        789,
        2026
    )

    noise_levels: tuple = (
        0.05,
        0.10,
        0.20,
        0.30
    )

    # -----
    # History
    # -----
    history_windows: tuple = (
        1,
        2,
        3,
        5,
        10
    )

    # -----
    # System
    # -----
    input_std: float = 0.50

    # -----
    # Output
    # -----
    output_dir: str = "hdd_benchmark_v1"

CFG = Config()

os.makedirs(CFG.output_dir, exist_ok=True)

```

```

np.random.seed(CFG.random_seed)

print("\nCONFIGURATION")
print("-" * 70)

for k, v in asdict(CFG).items():
    print(f"{k}: {v}")

# =====
# 2. ENVIRONMENT INFORMATION
# =====

ENVIRONMENT = {
    "python": sys.version,
    "platform": platform.platform(),
    "numpy": np.__version__,
    "pandas": pd.__version__,
    "sklearn": __import__("sklearn").__version__,
    "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
}

with open(
    os.path.join(CFG.output_dir, "environment.json"),
    "w"
) as f:
    json.dump(ENVIRONMENT, f, indent=2, default=str)

print("\nENVIRONMENT:")
print(json.dumps(ENVIRONMENT, indent=2))

# =====
# 3. SYSTEM GENERATORS
# =====

def generate_markov(
    length,
    burn_in,
    noise_std,
    rng
):
    """
    Fully observed Markovian system.

    
$$X_{t+1} = 0.70 X_t + 0.40 P_t + \text{noise}$$


    Conditioned on  $X_t$  and  $P_t$ ,
    additional history should not contain
    systematically relevant information.
    """

    total = length + burn_in

    x = np.zeros(total)
    p = rng.normal(
        0,
        CFG.input_std,
        total
    )

    x[0] = rng.normal()

    for t in range(total - 1):

        x[t + 1] = (
            0.70 * x[t]
            + 0.40 * p[t]

```

```

        + rng.normal(0, noise_std)
    )

    return x[burn_in:], p[burn_in:]

def generate_hidden_state(
    length,
    burn_in,
    noise_std,
    rng
):
    """
    System with a latent state.

     $z_{t+1} = 0.85 z_t + \text{input} + \text{noise}$ 

     $X_t = z_t + \text{observation noise}$ 

    The history of X contains information about
    the latent state that is not fully
    available in  $X_t$ .
    """

    total = length + burn_in

    z = np.zeros(total)
    x = np.zeros(total)

    p = rng.normal(
        0,
        CFG.input_std,
        total
    )

    z[0] = rng.normal()

    for t in range(total - 1):

        z[t + 1] = (
            0.85 * z[t]
            + 0.35 * p[t]
            + rng.normal(0, noise_std)
        )

        x[t] = (
            z[t]
            + rng.normal(0, noise_std)
        )

    x[-1] = (
        z[-1]
        + rng.normal(0, noise_std)
    )

    return x[burn_in:], p[burn_in:]

def generate_delay_line(
    length,
    burn_in,
    noise_std,
    rng
):
    """
    System with an explicit delay dependence.

     $X_{t+1}$  depends on:

```

```

X_t
X_{t-3}
P_t

```

This generates observable history dependence.
 """

```
total = length + burn_in + 5
```

```
x = np.zeros(total)
```

```
p = rng.normal(
    0,
    CFG.input_std,
    total
)
```

```
x[:5] = rng.normal(
    0,
    0.5,
    5
)
```

```
for t in range(4, total - 1):
```

```

    x[t + 1] = (
        0.55 * x[t]
        + 0.30 * x[t - 3]
        + 0.30 * p[t]
        + rng.normal(0, noise_std)
    )

```

```
start = burn_in + 5
```

```

return (
    x[start:start + length],
    p[start:start + length]
)

```

```

def generate_recurrent(
    length,
    burn_in,
    noise_std,
    rng
):

```

"""
 System with recurrent internal dynamics.

Hidden state:
 $h_{t+1} = \tanh(A h_t + B P_t + \text{noise})$

Observation:
 $X_t = C h_t + \text{observation noise}$

The history of X can recover information
 about the hidden recurrent state.
 """

```
total = length + burn_in
```

```

h = np.zeros((total, 2))
x = np.zeros(total)

```

```

p = rng.normal(
    0,
    CFG.input_std,
    total
)

```

```

    )

    A = np.array([
        [0.75, 0.20],
        [-0.15, 0.70]
    ])

    B = np.array([
        [0.35],
        [0.20]
    ])

    C = np.array([
        0.80,
        0.50
    ])

    h[0] = rng.normal(
        0,
        0.5,
        2
    )

    for t in range(total - 1):

        noise_vec = rng.normal(
            0,
            noise_std,
            2
        )

        h[t + 1] = np.tanh(
            A @ h[t]
            + B[:, 0] * p[t]
            + noise_vec
        )

        x[t] = (
            C @ h[t]
            + rng.normal(0, noise_std)
        )

    x[-1] = (
        C @ h[-1]
        + rng.normal(0, noise_std)
    )

    return x[burn_in:], p[burn_in:]

SYSTEM_GENERATORS = {
    "Markov": generate_markov,
    "Hidden State": generate_hidden_state,
    "Delay Line": generate_delay_line,
    "Recurrent": generate_recurrent,
}

# =====
# 4. TRAJECTORY GENERATION
# =====

def generate_dataset(
    system_name,
    n_trajectories,
    length,
    burn_in,
    noise_std,
    seed

```

```

):

    rng = np.random.default_rng(seed)

    trajectories = []

    generator = SYSTEM_GENERATORS[system_name]

    for i in range(n_trajectories):

        x, p = generator(
            length=length,
            burn_in=burn_in,
            noise_std=noise_std,
            rng=rng
        )

        trajectories.append({
            "trajectory_id": i,
            "x": np.asarray(x, dtype=float),
            "p": np.asarray(p, dtype=float)
        })

    return trajectories

# =====
# 5. SUPERVISED DATASET CONSTRUCTION
# =====

def make_supervised_dataset(
    trajectories,
    history_window
):
    """
    Builds:

    target:
        X_{t+1}

    present state:
        X_t
        P_t

    history:
        X_{t-1}, ..., X_{t-history_window}
        P_{t-1}, ..., P_{t-history_window}

    IMPORTANT:
    no future information enters the features.
    """

    X_present = []
    X_history = []
    y = []
    trajectory_ids = []

    for tr in trajectories:

        x = tr["x"]
        p = tr["p"]
        tid = tr["trajectory_id"]

        start = history_window
        end = len(x) - 1

        for t in range(start, end):

```

```

        # present state
        present = [
            x[t],
            p[t]
        ]

        # history
        history = []

        for lag in range(1, history_window + 1):

            history.extend([
                x[t - lag],
                p[t - lag]
            ])

            X_present.append(present)
            X_history.append(
                present + history
            )

            y.append(x[t + 1])
            trajectory_ids.append(tid)

    return (
        np.asarray(X_present),
        np.asarray(X_history),
        np.asarray(y),
        np.asarray(trajectory_ids)
    )

# =====
# 6. INTERNAL SPLIT BY TRAJECTORY
# =====

def trajectory_folds(
    trajectory_ids,
    n_folds
):

    unique_ids = np.unique(
        trajectory_ids
    )

    rng = np.random.default_rng(
        CFG.random_seed
    )

    shuffled = unique_ids.copy()

    rng.shuffle(shuffled)

    folds = np.array_split(
        shuffled,
        n_folds
    )

    return folds

# =====
# 7. ALPHA SELECTION
# =====

def select_alpha(

```

```

X,
y,
trajectory_ids,
alpha_grid,
n_folds
):
    """
    Selects alpha using only training data.

    Folds are defined by TRAJECTORY,
    preventing points from the same trajectory
    from appearing simultaneously in train and validation.
    """

    folds = trajectory_folds(
        trajectory_ids,
        n_folds
    )

    scores = {
        alpha: []
        for alpha in alpha_grid
    }

    unique_ids = np.unique(
        trajectory_ids
    )

    for alpha in alpha_grid:

        for validation_ids in folds:

            validation_ids = set(
                validation_ids.tolist()
            )

            train_mask = np.array([
                tid not in validation_ids
                for tid in trajectory_ids
            ])

            val_mask = ~train_mask

            if (
                train_mask.sum() == 0
                or val_mask.sum() == 0
            ):
                continue

            model = Pipeline([
                (
                    "scale",
                    StandardScaler()
                ),
                (
                    "ridge",
                    Ridge(
                        alpha=alpha,
                        fit_intercept=True
                    )
                )
            ])

            model.fit(
                X[train_mask],
                y[train_mask]
            )

            pred = model.predict(

```

```

        X[val_mask]
    )

    mse = np.mean(
        (y[val_mask] - pred) ** 2
    )

    scores[alpha].append(mse)

mean_scores = {
    alpha: np.mean(values)
    for alpha, values in scores.items()
    if len(values) > 0
}

best_alpha = min(
    mean_scores,
    key=mean_scores.get
)

return best_alpha, mean_scores

# =====
# 8. FINAL FIT
# =====

def fit_final_model(
    X,
    y,
    alpha
):
    model = Pipeline([
        (
            "scale",
            StandardScaler()
        ),
        (
            "ridge",
            Ridge(
                alpha=alpha,
                fit_intercept=True
            )
        )
    ])

    model.fit(X, y)

    return model

# =====
# 9. PER-TRAJECTORY METRICS
# =====

def trajectory_losses(
    model,
    X,
    y,
    trajectory_ids
):
    pred = model.predict(X)

    df = pd.DataFrame({
        "trajectory_id": trajectory_ids,
        "y": y,

```

```

        "pred": pred
    })

    df["sq_error"] = (
        df["y"] - df["pred"]
    ) ** 2

    losses = (
        df.groupby("trajectory_id")
        ["sq_error"]
        .mean()
    )

    return losses.values, pred

# =====
# 10. PAIRED BOOTSTRAP BY TRAJECTORY
# =====

def bootstrap_difference(
    loss_present,
    loss_history,
    n_boot,
    seed
):
    """
    Trajectory-level paired bootstrap.

    The resampling unit is the trajectory,
    NOT each individual time point.

    This preserves the internal dependence
    within each trajectory.
    """

    rng = np.random.default_rng(seed)

    diff = (
        loss_present
        - loss_history
    )

    n = len(diff)

    boot = np.empty(n_boot)

    for i in range(n_boot):

        idx = rng.integers(
            0,
            n,
            size=n
        )

        boot[i] = np.mean(
            diff[idx]
        )

    lower = np.quantile(
        boot,
        0.025
    )

    upper = np.quantile(
        boot,
        0.975
    )

```

```

        return (
            np.mean(diff),
            lower,
            upper,
            boot
        )

# =====
# 11. PAIRED PERMUTATION TEST
# =====

def paired_permutation_test(
    loss_present,
    loss_history,
    n_perm,
    seed
):
    """
    Paired permutation test.

    Under  $H_0$ , within each trajectory,
    the identity of which model obtained
    which error can be permuted.

     $H_1$ :
        MSE_present > MSE_history
    """

    rng = np.random.default_rng(seed)

    diff = (
        loss_present
        - loss_history
    )

    observed = np.mean(diff)

    n = len(diff)

    extreme = 0

    for _ in range(n_perm):

        signs = rng.choice(
            [-1, 1],
            size=n
        )

        permuted = np.mean(
            diff * signs
        )

        if permuted >= observed:
            extreme += 1

    p_value = (
        extreme + 1
    ) / (
        n_perm + 1
    )

    return observed, p_value

# =====

```

```

# 12. HDD-I CLASSIFICATION
# =====

def classify_construct_I(
    mean_difference,
    ci_lower,
    ci_upper,
    relative_improvement,
    p_value,
    practical_threshold
):
    """
    Pre-specified criterion.

    '+':
        positive effect
        + CI does not cross zero
        + relative improvement >= threshold
        + p < 0.05

    '-':
        evidence against a relevant improvement:
        upper CI <= 0
        OR relative improvement below threshold

    UE:
        insufficient data / inconclusive result
    """

    if (
        ci_lower > 0
        and relative_improvement >= practical_threshold
        and p_value < CFG.alpha_stat
    ):
        return "+"

    if (
        ci_upper <= 0
        or relative_improvement < 0
    ):
        return "-"

    return "UE"

# =====
# 13. PLACEBO HISTORY MODEL
# =====

def make_placebo_history(
    X_history,
    rng
):
    """
    Keeps the SAME number of historical features,
    but breaks their temporal relationship with the target.

    This tests whether an improvement simply appears
    because the model has more parameters/features.

    Present-state features remain intact.
    """

    X_placebo = X_history.copy()

    if X_history.shape[1] <= 2:
        return X_placebo

    historical = X_placebo[:, 2:].copy()

```

```

    permutation = rng.permutation(
        historical.shape[0]
    )

    historical = historical[
        permutation
    ]

    X_placebo[:, 2:] = historical

    return X_placebo

# =====
# 14. MAIN EXPERIMENT
# =====

def run_single_system(
    system_name,
    noise_std,
    seed
):

    print("\n" + "=" * 70)
    print(f"SYSTEM: {system_name}")
    print(f"Noise: {noise_std}")
    print("=" * 70)

    train = generate_dataset(
        system_name=system_name,
        n_trajectories=CFG.n_train_trajectories,
        length=CFG.trajectory_length,
        burn_in=CFG.burn_in,
        noise_std=noise_std,
        seed=seed
    )

    test = generate_dataset(
        system_name=system_name,
        n_trajectories=CFG.n_test_trajectories,
        length=CFG.trajectory_length,
        burn_in=CFG.burn_in,
        noise_std=noise_std,
        seed=seed + 100000
    )

    results = []

    detailed_predictions = {}

    # -----
    # different history windows
    # -----

    for history_window in CFG.history_windows:

        print(
            f"\n History = {history_window}"
        )

        (
            Xp_train,
            Xh_train,
            y_train,
            tid_train
        ) = make_supervised_dataset(
            train,

```

```

        history_window
    )

    (
        Xp_test,
        Xh_test,
        y_test,
        tid_test
    ) = make_supervised_dataset(
        test,
        history_window
    )

# -----
# Model M0
# -----

alpha_present, _ = select_alpha(
    Xp_train,
    y_train,
    tid_train,
    CFG.alpha_grid,
    CFG.inner_folds
)

model_present = fit_final_model(
    Xp_train,
    y_train,
    alpha_present
)

# -----
# Model M1
# -----

alpha_history, _ = select_alpha(
    Xh_train,
    y_train,
    tid_train,
    CFG.alpha_grid,
    CFG.inner_folds
)

model_history = fit_final_model(
    Xh_train,
    y_train,
    alpha_history
)

# -----
# Predictions
# -----

loss_p, pred_p = trajectory_losses(
    model_present,
    Xp_test,
    y_test,
    tid_test
)

loss_h, pred_h = trajectory_losses(
    model_history,
    Xh_test,
    y_test,
    tid_test
)

# -----
# Bootstrap
# -----

```

```

(
    delta,
    ci_lower,
    ci_upper,
    boot
) = bootstrap_difference(
    loss_p,
    loss_h,
    CFG.bootstrap_reps,
    seed + history_window
)

# -----
# Permutation
# -----

(
    observed_perm,
    p_value
) = paired_permutation_test(
    loss_p,
    loss_h,
    CFG.permutation_reps,
    seed + 1000 + history_window
)

# -----
# Global MSE
# -----

mse_present = np.mean(
    loss_p
)

mse_history = np.mean(
    loss_h
)

relative_improvement = (
    mse_present - mse_history
) / mse_present

# -----
# R2
# -----

ss_total = np.sum(
    (
        y_test
        - np.mean(y_test)
    ) ** 2
)

ss_present = np.sum(
    (
        y_test
        - pred_p
    ) ** 2
)

ss_history = np.sum(
    (
        y_test
        - pred_h
    ) ** 2
)

r2_present = (
    1

```

```

        - ss_present / ss_total
    )

    r2_history = (
        1
        - ss_history / ss_total
    )

    classification = classify_construct_I(
        delta,
        ci_lower,
        ci_upper,
        relative_improvement,
        p_value,
        CFG.practical_relative_threshold
    )

    result = {
        "System": system_name,
        "Noise": noise_std,
        "History_Window": history_window,
        "Alpha_Present": alpha_present,
        "Alpha_History": alpha_history,
        "MSE_Present": mse_present,
        "MSE_History": mse_history,
        "Delta_MSE": delta,
        "Relative_Improvement": relative_improvement,
        "R2_Present": r2_present,
        "R2_History": r2_history,
        "CI95_Lower": ci_lower,
        "CI95_Upper": ci_upper,
        "Permutation_P": p_value,
        "HDD_Construct_I": classification
    }

    results.append(result)

    detailed_predictions[
        history_window
    ] = {
        "y": y_test,
        "pred_present": pred_p,
        "pred_history": pred_h,
        "loss_present": loss_p,
        "loss_history": loss_h
    }

    print(
        f"    MSE present:          {mse_present:.8f}"
    )

    print(
        f"    MSE history:           {mse_history:.8f}"
    )

    print(
        f"    \u0394MSE:              {delta:.8f}"
    )

    print(
        f"    Relative improvement:  "
        f"{100 * relative_improvement:.3f}%"
    )

    print(
        f"    95% CI \u0394MSE:      "
        f"{[ci_lower:.8f], [ci_upper:.8f]}"
    )

    print(

```

```

        f"    Permutation p:      {p_value:.6f}"
    )

    print(
        f"    HDD-I:                {classification}"
    )

    return (
        pd.DataFrame(results),
        detailed_predictions
    )

# =====
# 15. MAIN EXECUTION
# =====

all_results = []
all_predictions = {}

start_time = time.time()

for system_name in SYSTEM_GENERATORS:

    result_df, predictions = run_single_system(
        system_name=system_name,
        noise_std=CFG.noise_std,
        seed=CFG.random_seed
    )

    all_results.append(result_df)

    all_predictions[
        system_name
    ] = predictions

results_df = pd.concat(
    all_results,
    ignore_index=True
)

elapsed = time.time() - start_time

print("\n")
print("=" * 70)
print("MAIN RUN COMPLETE")
print("=" * 70)

print(
    f"Time: {elapsed / 60:.2f} minutes"
)

# =====
# 16. MAIN RESULT
# =====

main_window = max(
    CFG.history_windows
)

main_results = (
    results_df[
        results_df["History_Window"]
        == main_window
    ]

```

```

        .copy()
    )

    print("\n")
    print("=" * 70)
    print(
        f"MAIN RESULT \u2014 HISTORY = {main_window}"
    )
    print("=" * 70)

    display_columns = [
        "System",
        "MSE_Present",
        "MSE_History",
        "Delta_MSE",
        "Relative_Improvement",
        "CI95_Lower",
        "CI95_Upper",
        "Permutation_P",
        "HDD_Construct_I"
    ]

    display(
        main_results[
            display_columns
        ].round(6)
    )

# =====
# 17. HISTORY DEPENDENCE PROFILE \u0394L(\u03c4)
# =====

    print("\n")
    print("=" * 70)
    print("HISTORY DEPENDENCE PROFILE")
    print("=" * 70)

    plt.figure(
        figsize=(11, 7)
    )

    for system_name in SYSTEM_GENERATORS:

        subset = results_df[
            results_df["System"]
            == system_name
        ]

        plt.plot(
            subset["History_Window"],
            100 * subset["Relative_Improvement"],
            marker="o",
            linewidth=2,
            label=system_name
        )

    plt.axhline(
        100 * CFG.practical_relative_threshold,
        color="black",
        linestyle="--",
        alpha=0.7,
        label="Practical threshold"
    )

    plt.axhline(
        0,
        color="gray",
        linestyle=":",
        alpha=0.7
    )

```

```

)

plt.xlabel(
    "Maximum history window \u03c4"
)

plt.ylabel(
    "Relative improvement (%)"
)

plt.title(
    "HDD Construct I \u2014 History dependence profile"
)

plt.legend()
plt.grid(alpha=0.25)

plt.tight_layout()

plt.savefig(
    os.path.join(
        CFG.output_dir,
        "history_dependence_profile.png"
    ),
    dpi=200
)

plt.show()

# =====
# 18. CAPACITY CONTROL \u2014 PLACEBO HISTORY
# =====

def run_capacity_control(
    system_name,
    noise_std,
    history_window,
    seed
):

    print("\n")
    print("=" * 70)
    print(
        f"CAPACITY CONTROL \u2014 {system_name}"
    )
    print("=" * 70)

    train = generate_dataset(
        system_name,
        CFG.n_train_trajectories,
        CFG.trajectory_length,
        CFG.burn_in,
        noise_std,
        seed
    )

    test = generate_dataset(
        system_name,
        CFG.n_test_trajectories,
        CFG.trajectory_length,
        CFG.burn_in,
        noise_std,
        seed + 100000
    )

    (
        Xp_train,
        Xh_train,

```

```

        y_train,
        tid_train
    ) = make_supervised_dataset(
        train,
        history_window
    )

    (
        Xp_test,
        Xh_test,
        y_test,
        tid_test
    ) = make_supervised_dataset(
        test,
        history_window
    )

    # present model
    alpha_p, _ = select_alpha(
        Xp_train,
        y_train,
        tid_train,
        CFG.alpha_grid,
        CFG.inner_folds
    )

    model_p = fit_final_model(
        Xp_train,
        y_train,
        alpha_p
    )

    # real history model
    alpha_h, _ = select_alpha(
        Xh_train,
        y_train,
        tid_train,
        CFG.alpha_grid,
        CFG.inner_folds
    )

    model_h = fit_final_model(
        Xh_train,
        y_train,
        alpha_h
    )

    # placebo history
    rng = np.random.default_rng(
        seed + 9999
    )

    Xh_train_placebo = make_placebo_history(
        Xh_train,
        rng
    )

    Xh_test_placebo = make_placebo_history(
        Xh_test,
        rng
    )

    alpha_placebo, _ = select_alpha(
        Xh_train_placebo,
        y_train,
        tid_train,
        CFG.alpha_grid,
        CFG.inner_folds
    )

    model_placebo = fit_final_model(

```

```

        Xh_train_placebo,
        y_train,
        alpha_placebo
    )

    loss_p, _ = trajectory_losses(
        model_p,
        Xp_test,
        y_test,
        tid_test
    )

    loss_h, _ = trajectory_losses(
        model_h,
        Xh_test,
        y_test,
        tid_test
    )

    loss_placebo, _ = trajectory_losses(
        model_placebo,
        Xh_test_placebo,
        y_test,
        tid_test
    )

    real_improvement = (
        np.mean(loss_p)
        - np.mean(loss_h)
    ) / np.mean(loss_p)

    placebo_improvement = (
        np.mean(loss_p)
        - np.mean(loss_placebo)
    ) / np.mean(loss_p)

    return {
        "System": system_name,
        "Real_Improvement": real_improvement,
        "Placebo_Improvement": placebo_improvement,
        "Real_MSE": np.mean(loss_h),
        "Placebo_MSE": np.mean(loss_placebo)
    }

capacity_results = []

for system_name in SYSTEM_GENERATORS:

    capacity_results.append(
        run_capacity_control(
            system_name,
            CFG.noise_std,
            main_window,
            CFG.random_seed
        )
    )

capacity_df = pd.DataFrame(
    capacity_results
)

print("\nCAPACITY CONTROL")

display(
    capacity_df.round(6)
)

```

```

capacity_df.to_csv(
    os.path.join(
        CFG.output_dir,
        "capacity_control.csv"
    ),
    index=False
)

# =====
# 19. STATE-RECONSTRUCTION DIAGNOSTIC
# =====

def state_reconstruction_diagnostic(
    system_name,
    noise_std,
    history_window,
    seed
):
    """
    State-reconstruction diagnostic.

    This is NOT presented as proof of "irreducible history".

    The idea is to check whether a compressed
    representation of the history preserves its
    predictive advantage.

    We use PCA trained ONLY on the training data.

    This tests a weaker hypothesis:

        does the advantage of history disappear
        when history is compressed into a low-dimensional
        representation?

    If yes:
        this suggests that history may be providing
        information about a latent/reconstructible state.

    If no:
        the dependence may be more complex.

    This does NOT prove ontological irreducibility.
    """

    train = generate_dataset(
        system_name,
        CFG.n_train_trajectories,
        CFG.trajectory_length,
        CFG.burn_in,
        noise_std,
        seed
    )

    test = generate_dataset(
        system_name,
        CFG.n_test_trajectories,
        CFG.trajectory_length,
        CFG.burn_in,
        noise_std,
        seed + 100000
    )

    (
        Xp_train,
        Xh_train,
        y_train,
        tid_train
    )

```

```

) = make_supervised_dataset(
    train,
    history_window
)

(
    Xp_test,
    Xh_test,
    y_test,
    tid_test
) = make_supervised_dataset(
    test,
    history_window
)

# history block only
H_train = Xh_train[:, 2:]
H_test = Xh_test[:, 2:]

# cap dimensionality in a pre-defined way
n_components = min(
    3,
    H_train.shape[1]
)

scaler = StandardScaler()

H_train_scaled = scaler.fit_transform(
    H_train
)

H_test_scaled = scaler.transform(
    H_test
)

pca = PCA(
    n_components=n_components
)

S_train = pca.fit_transform(
    H_train_scaled
)

S_test = pca.transform(
    H_test_scaled
)

# present state + reconstructed state
Xs_train = np.column_stack([
    Xp_train,
    S_train
])

Xs_test = np.column_stack([
    Xp_test,
    S_test
])

alpha_s, _ = select_alpha(
    Xs_train,
    y_train,
    tid_train,
    CFG.alpha_grid,
    CFG.inner_folds
)

model_s = fit_final_model(
    Xs_train,
    y_train,

```

```

        alpha_s
    )

    loss_s, _ = trajectory_losses(
        model_s,
        Xs_test,
        y_test,
        tid_test
    )

    mse_s = np.mean(loss_s)

    return {
        "System": system_name,
        "MSE_Reconstructed_State": mse_s,
        "Explained_Variance_PCA": np.sum(
            pca.explained_variance_ratio_
        )
    }

reconstruction_results = []

for system_name in SYSTEM_GENERATORS:

    reconstruction_results.append(
        state_reconstruction_diagnostic(
            system_name,
            CFG.noise_std,
            main_window,
            CFG.random_seed
        )
    )

reconstruction_df = pd.DataFrame(
    reconstruction_results
)

print("\n")
print("=" * 70)
print("STATE-RECONSTRUCTION DIAGNOSTIC")
print("=" * 70)

display(
    reconstruction_df.round(6)
)

reconstruction_df.to_csv(
    os.path.join(
        CFG.output_dir,
        "state_reconstruction.csv"
    ),
    index=False
)

# =====
# 20. NOISE ROBUSTNESS
# =====

noise_results = []

print("\n")
print("=" * 70)
print("NOISE ROBUSTNESS ANALYSIS")
print("=" * 70)

for noise in CFG.noise_levels:

```

```

print(
    f"\nNoise = {noise}"
)

for system_name in SYSTEM_GENERATORS:

    df, _ = run_single_system(
        system_name,
        noise,
        CFG.random_seed
    )

    selected = df[
        df["History_Window"]
        == main_window
    ].copy()

    noise_results.append(
        selected
    )

noise_df = pd.concat(
    noise_results,
    ignore_index=True
)

noise_df.to_csv(
    os.path.join(
        CFG.output_dir,
        "noise_robustness.csv"
    ),
    index=False
)

# =====
# 21. ROBUSTNESS TO MULTIPLE SEEDS
# =====

seed_results = []

print("\n")
print("=" * 70)
print("ROBUSTNESS TO MULTIPLE SEEDS")
print("=" * 70)

for seed in CFG.robustness_seeds:

    print(
        f"\nSeed = {seed}"
    )

    for system_name in SYSTEM_GENERATORS:

        df, _ = run_single_system(
            system_name,
            CFG.noise_std,
            seed
        )

        selected = df[
            df["History_Window"]
            == main_window
        ].copy()

        selected["Seed"] = seed

```

```

        seed_results.append(
            selected
        )

seed_df = pd.concat(
    seed_results,
    ignore_index=True
)

seed_df.to_csv(
    os.path.join(
        CFG.output_dir,
        "multi_seed_robustness.csv"
    ),
    index=False
)

# =====
# 22. CONSISTENCY ACROSS SEEDS
# =====

seed_summary = []

for system_name in SYSTEM_GENERATORS:

    subset = seed_df[
        seed_df["System"]
        == system_name
    ]

    positive_fraction = np.mean(
        subset["HDD_Construct_I"]
        == "+"
    )

    mean_improvement = np.mean(
        subset["Relative_Improvement"]
    )

    std_improvement = np.std(
        subset["Relative_Improvement"],
        ddof=1
    )

    seed_summary.append({
        "System": system_name,
        "Positive_Fraction": positive_fraction,
        "Mean_Relative_Improvement":
            mean_improvement,
        "SD_Relative_Improvement":
            std_improvement
    })

seed_summary_df = pd.DataFrame(
    seed_summary
)

print("\n")
print("=" * 70)
print("ROBUSTNESS SUMMARY")
print("=" * 70)

display(
    seed_summary_df.round(6)
)

```

```

)

# =====
# 23. BEHAVIORAL EQUIVALENCE BENCHMARK
# =====

print("\n")
print("=" * 70)
print("BEHAVIORAL EQUIVALENCE BENCHMARK")
print("=" * 70)

print(
    """
    This test matters for interpreting HDD.

    Delay Line and Recurrent may have different
    internal mechanisms, but Construct I should not
    attempt to infer architecture from a simple
    predictive improvement.

    Therefore:

    Positive Construct I
      does NOT imply
    positive Construct III.
    """
)

# =====
# 24. GROUND TRUTH
# =====

ground_truth = pd.DataFrame({

    "System": [
        "Markov",
        "Hidden State",
        "Delay Line",
        "Recurrent"
    ],

    "Expected_Construct_I": [
        "-",
        "+",
        "+",
        "+"
    ],

    "Reason": [
        "Observed state is sufficient",
        "History reveals latent state",
        "Dynamics explicitly depend on delay",
        "History helps infer hidden recurrent state"
    ]
})

final_main = main_results.merge(
    ground_truth,
    on="System",
    how="left"
)

final_main[
    "Correct_vs_Ground_Truth"
] = (
    final_main[
        "HDD_Construct_I"

```

```

    ]
    ==
    final_main[
        "Expected_Construct_I"
    ]
)

print("\n")
print("=" * 70)
print("COMPARISON WITH GROUND TRUTH")
print("=" * 70)

display(
    final_main[
        [
            "System",
            "HDD_Construct_I",
            "Expected_Construct_I",
            "Correct_vs_Ground_Truth"
        ]
    ]
)

# =====
# 25. PERFORMANCE MATRIX
# =====

accuracy = np.mean(
    final_main[
        "Correct_vs_Ground_Truth"
    ]
)

print(
    f"\nAccuracy against ground truth: "
    f"{100 * accuracy:.1f}%"
)

# =====
# 26. ROC-LIKE CURVE / EFFECT SIZE
# =====

plt.figure(
    figsize=(10, 6)
)

colors = {
    "Markov": "black",
    "Hidden State": "blue",
    "Delay Line": "orange",
    "Recurrent": "red"
}

for system_name in SYSTEM_GENERATORS:

    subset = results_df[
        results_df["System"]
        == system_name
    ]

    plt.errorbar(
        subset["History_Window"],
        100 * subset["Relative_Improvement"],
        fmt="o-",
        capsize=4,
        linewidth=2,
        label=system_name,

```

```

        color=colors[system_name]
    )

plt.axhline(
    0,
    color="gray",
    linestyle=":"
)

plt.axhline(
    100 * CFG.practical_relative_threshold,
    color="black",
    linestyle="--",
    label="Practical threshold"
)

plt.xlabel(
    "Maximum history \u03c4"
)

plt.ylabel(
    "Relative improvement (%)"
)

plt.title(
    "Effect of history on prediction"
)

plt.grid(
    alpha=0.25
)

plt.legend()

plt.tight_layout()

plt.savefig(
    os.path.join(
        CFG.output_dir,
        "effect_size_profile.png"
    ),
    dpi=200
)

plt.show()

# =====
# 27. FULL FINAL TABLE
# =====

final_results = results_df.merge(
    ground_truth,
    on="System",
    how="left"
)

final_results[
    "Correct_vs_Ground_Truth"
] = np.where(
    final_results["HDD_Construct_I"]
    == final_results["Expected_Construct_I"],
    True,
    False
)

final_results.to_csv(
    os.path.join(
        CFG.output_dir,

```

```

        "HDD_Construct_I_full_results.csv"
    ),
    index=False
)

main_results.to_csv(
    os.path.join(
        CFG.output_dir,
        "HDD_Construct_I_main_results.csv"
    ),
    index=False
)

# =====
# 28. REPRODUCIBILITY MANIFEST
# =====

manifest = {
    "benchmark_version": "HDD Benchmark v1.0",
    "construct": "I",
    "definition":
        "History improves out-of-sample prediction "
        "beyond measured present state and current input",

    "configuration": asdict(CFG),

    "ground_truth": ground_truth.to_dict(
        orient="records"
    ),

    "environment": ENVIRONMENT,

    "files": [
        "HDD_Construct_I_full_results.csv",
        "HDD_Construct_I_main_results.csv",
        "capacity_control.csv",
        "state_reconstruction.csv",
        "noise_robustness.csv",
        "multi_seed_robustness.csv",
        "history_dependence_profile.png",
        "effect_size_profile.png",
        "environment.json"
    ],

    "important_interpretation":
        "Positive Construct I does not establish memory "
        "mechanism, recurrence, self-reference, self-modeling "
        "or consciousness."
}

with open(
    os.path.join(
        CFG.output_dir,
        "manifest.json"
    ),
    "w"
) as f:
    json.dump(
        manifest,
        f,
        indent=2,
        default=str
    )

# =====
# 29. AUTOMATIC REPORT
# =====

```

```

report_lines = []

report_lines.append(
    "HDD BENCHMARK v1.0"
)

report_lines.append(
    "Construct I \u2014 History-Dependent Predictive Structure"
)

report_lines.append(
    ""
)

report_lines.append(
    "MAIN RESULT"
)

for _, row in main_results.iterrows():

    report_lines.append(
        f"{row['System']}: "
        f"HDD-I={row['HDD_Construct_I']}, "
        f"improvement="
        f"{100 * row['Relative_Improvement']:.3f}%, "
        f"CI95=["
        f"{row['CI95_Lower']:.8f}, "
        f"{row['CI95_Upper']:.8f}], "
        f"p="
        f"{row['Permutation_P']:.6f}"
    )

report_lines.append(
    ""
)

report_lines.append(
    "GROUND TRUTH"
)

for _, row in ground_truth.iterrows():

    report_lines.append(
        f"{row['System']}: "
        f"expected={row['Expected_Construct_I']}"
    )

report_lines.append(
    ""
)

report_lines.append(
    f"Preliminary benchmark accuracy: "
    f"{100 * accuracy:.1f}%"
)

report_lines.append(
    ""
)

report_lines.append(
    "INTERPRETATION:"
)

report_lines.append(
    "A positive Construct I classification means only "
    "that historical information improved out-of-sample "

```

```

        "prediction relative to the specified observed present "
        "state and current input."
    )

    report_lines.append(
        "It does not establish a memory mechanism, recurrence, "
        "self-reference, self-modeling, agency or consciousness."
    )

    report_lines.append(
        ""
    )

    report_lines.append(
        "LIMITATION:"
    )

    report_lines.append(
        "This benchmark validates the computational "
        "discrimination of Construct I under the specified "
        "observation and model class. It does not validate "
        "the complete HDD framework."
    )

    report_text = "\n".join(
        report_lines
    )

    print("\n")
    print("=" * 70)
    print("REPORT")
    print("=" * 70)

    print(report_text)

    with open(
        os.path.join(
            CFG.output_dir,
            "REPORT.txt"
        ),
        "w"
    ) as f:
        f.write(report_text)

# =====
# 30. FINAL ZIP
# =====

import shutil

zip_path = shutil.make_archive(
    CFG.output_dir,
    "zip",
    CFG.output_dir
)

print("\n")
print("=" * 70)
print("BENCHMARK FINISHED")
print("=" * 70)

print(
    f"\nFiles saved to:\n"
    f"{os.path.abspath(CFG.output_dir)}"
)

print(
    f"\nZIP package:\n"

```

```

        f"{zip_path}"
    )

print("\n")
print("=" * 70)
print("SCIENTIFIC INTERPRETATION")
print("=" * 70)

print(
    """
The benchmark was designed to answer only:

    Does history improve prediction of the future
    beyond the observed present state?

It does NOT answer:

    Is history memory?
    Does recurrence exist?
    Does self-reference exist?
    Does a self-model exist?
    Does consciousness exist?

Those questions require other experiments.

The correct result must be interpreted
relative to the observation model, intervention
protocol, and model class used.

The Hidden State system is especially important:
a positive result on it does NOT mean the system
has an explicit memory. History may simply
reveal information about a latent variable.

Therefore, the benchmark tests precisely one of
HDD's central methodological theses:

    "predictive history is not automatically
    evidence of a specific mechanism."
    """
)

print("\n")
print("=" * 70)
print("END")
print("=" * 70)

```